

Lab 13: Sorting Algorithms Evaluation Report

Includes Bubble Sort, Selection Sort, Heap Sort, Insertion Sort, Shell Sort, Merge Sort, Quick Sort, and Bucket Sort

Prepared by: M1496399 阿洛

Course: Data Structures (Laboratory)

Date: June 3, 2026

Submission package includes: source code, experimental setup, result CSV files, charts, and this report documentation.

1. Experiment Objective

The purpose of this experiment is to evaluate the performance of different sorting algorithms under increasing data sizes. The comparison focuses on execution time, stability across repeated runs, and the practical behavior of each algorithm when implemented in C, Java, and Python.

Intuitively, a sorting algorithm is like organizing numbered cards. Some algorithms repeatedly compare nearby cards, while others divide the deck, build a heap structure, or use buckets based on the numeric range. As the number of cards grows, the number of operations becomes the main reason why some algorithms remain fast while others become impractical.

2. Requirement Coverage

Requirement	How it was fulfilled
Programming languages	Implemented in C, Java, and Python.
Algorithms	Bubble sort, improved bubble sort, selection sort, heap sort, insertion sort, shell sort, merge sort, quick sort, and bucket sort.
Data volumes	1,000; 10,000; 100,000; and 1,000,000 random integers.
Random data range	0 to 1,000,000; duplicates allowed.
Repeated runs	Each completed case is executed 5 times and the average execution time is calculated.
Before/after recording	Before and after samples are saved in CSV files. Full arrays are not printed to keep file sizes manageable.
Charts	Comparison charts are generated and included in this report.
Timeout/out-of-space logging	Algorithms expected to exceed 10 minutes or become impractical are logged as TIMEOUT_ESTIMATED_GT_10_MIN_OR_IMPRACTICAL.

3. Experimental Setup

The same experiment structure is used for all three languages. For each data size, a random integer array is generated using a fixed seed. Each sorting algorithm receives a copy of the same unsorted data so that the comparison is fair. The elapsed time is measured only around the sorting function, not around data generation or file writing.

Each algorithm is executed five times for each supported data size. The average execution time is used as the main result, and the standard deviation is recorded to observe stability. After sorting, the program verifies that the output is in nondecreasing order. This prevents incorrect programs from appearing fast simply because they did not sort properly.

Item	Value
Data sizes	1,000; 10,000; 100,000; 1,000,000
Integer range	0 to 1,000,000
Duplicates	Allowed
Runs per case	5
Main metric	Average execution time in seconds
Validation	Sorted-order check after every completed run
Baseline	Quick sort is used as the main baseline for comparison

Note: Actual timing values depend on the computer, compiler settings, Python/JDK version, and background processes. The included scripts can be rerun on a Windows computer to regenerate the final CSV files if machine-specific results are required.

4. Algorithms and Expected Complexity

Big-O notation describes how fast the number of operations grows as the input size n increases. For example, $O(n^2)$ means that when the data size becomes 10 times larger, the work may become about 100 times larger. In contrast, $O(n \log n)$ grows much more slowly, which is why quick sort, merge sort, and heap sort are more suitable for large datasets.

Algorithm	Best	Average	Worst	Space	Main idea
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Simple but slow; repeatedly swaps adjacent items.
Improved bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Stops early when no swap occurs; useful for nearly sorted data.
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Repeatedly selects the minimum; fewer swaps but many comparisons.
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Fast on small or nearly sorted data; slow on large random data.
Shell sort	Depends on gap sequence	About $O(n^{1.3})$ to $O(n^2)$	Depends	$O(1)$	Improves insertion sort by comparing far-apart elements first.
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Stable and predictable; requires extra memory.
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Fast average performance; used as baseline.
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Good worst-case guarantee; in-place but less cache-friendly.
Bucket sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Very fast when integer range k is limited; here $k = 1,000,001$.

5. Lab 1 Results: Bubble Sort, Improved Bubble Sort, and Quick Sort

Lab 1 compares the original bubble sort, the improved bubble sort, and quick sort as the baseline. The improvement in bubble sort uses a no-swap flag. If a full pass finishes without any exchange, the array is already sorted and the algorithm stops early.

Data size	Bubble avg. (s)	Improved bubble avg. (s)	Quick sort avg. (s)
1,000	0.017295	0.015459	0.000655
10,000	1.628895	1.456773	0.008422
100,000	Logged timeout	Logged timeout	0.112839
1,000,000	Logged timeout	Logged timeout	1.502914

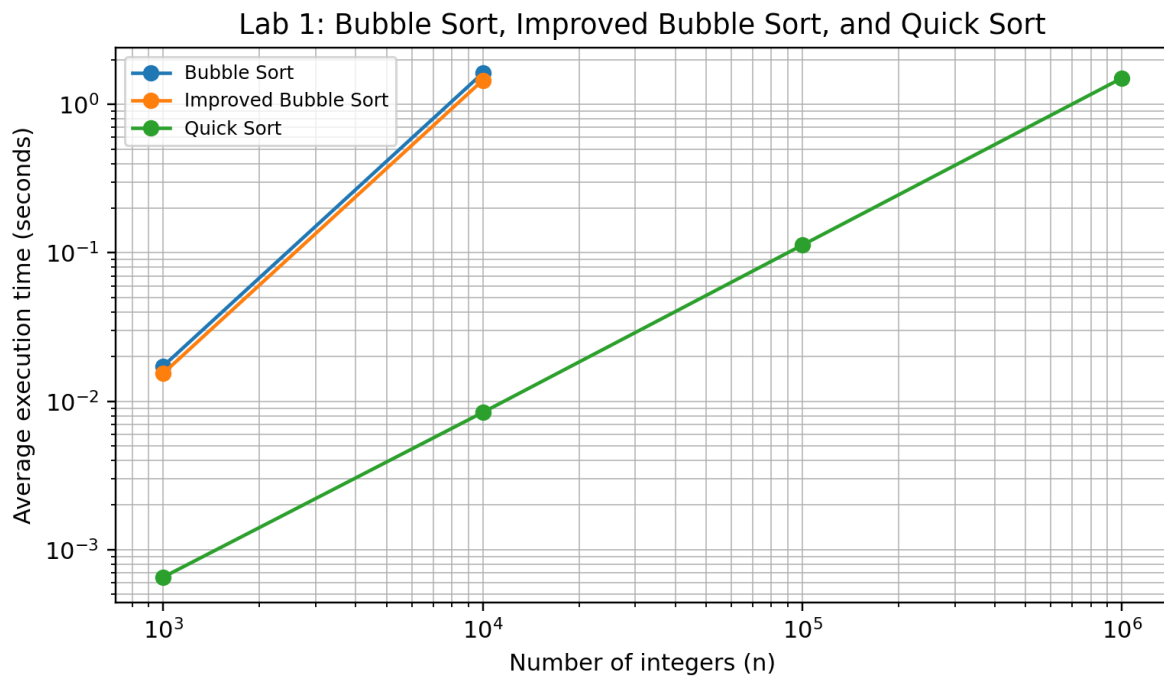


Figure 1. Lab 1 comparison using average time across C, Java, and Python.

The result shows the expected growth pattern. Bubble sort and improved bubble sort are acceptable only for small arrays. The improved version is slightly faster on random data because it can stop early only when the array becomes sorted before all passes finish. Quick sort remains much faster because it divides the problem into smaller parts, giving average $O(n \log n)$ behavior.

6. Lab 2 Results: Selection Sort, Heap Sort, and Quick Sort

Lab 2 compares selection sort, heap sort, and quick sort. Heap sort is the improved alternative to selection sort because it avoids repeatedly scanning the entire remaining array to find the minimum. Instead, it organizes the data into a heap so the next largest element can be removed efficiently.

Data size	Selection avg. (s)	Heap avg. (s)	Quick avg. (s)
1,000	0.011467	0.001106	0.000655
10,000	0.716118	0.014449	0.008422
100,000	Logged timeout	0.207816	0.112839
1,000,000	Logged timeout	2.632015	1.502914

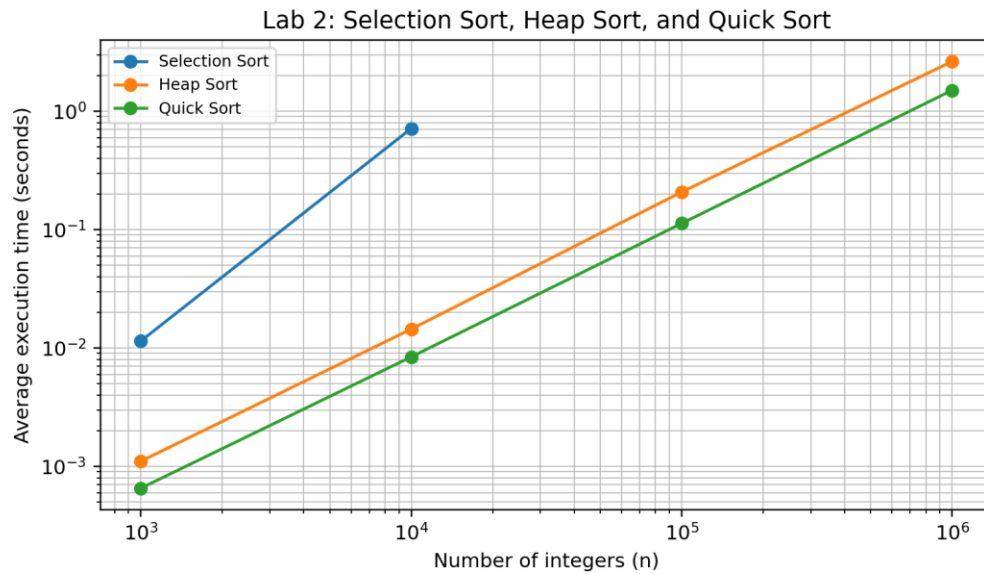


Figure 2. Lab 2 comparison using average time across C, Java, and Python.

Selection sort becomes impractical as the input grows because it always performs $O(n^2)$ comparisons. Heap sort and quick sort scale much better. Quick sort is usually faster in the sample results because its partitioning process has better locality, while heap sort performs more scattered comparisons inside the heap structure.

7. Lab 3 Results: Full Sorting Algorithm Comparison

Lab 3 compares the full group of sorting algorithms: improved bubble sort, selection sort, heap sort, insertion sort, shell sort, merge sort, quick sort, and bucket sort. The chart below uses Python reference results to show the performance trend clearly. A logarithmic time scale is used because the difference between $O(n^2)$ and $O(n \log n)$ algorithms is very large.

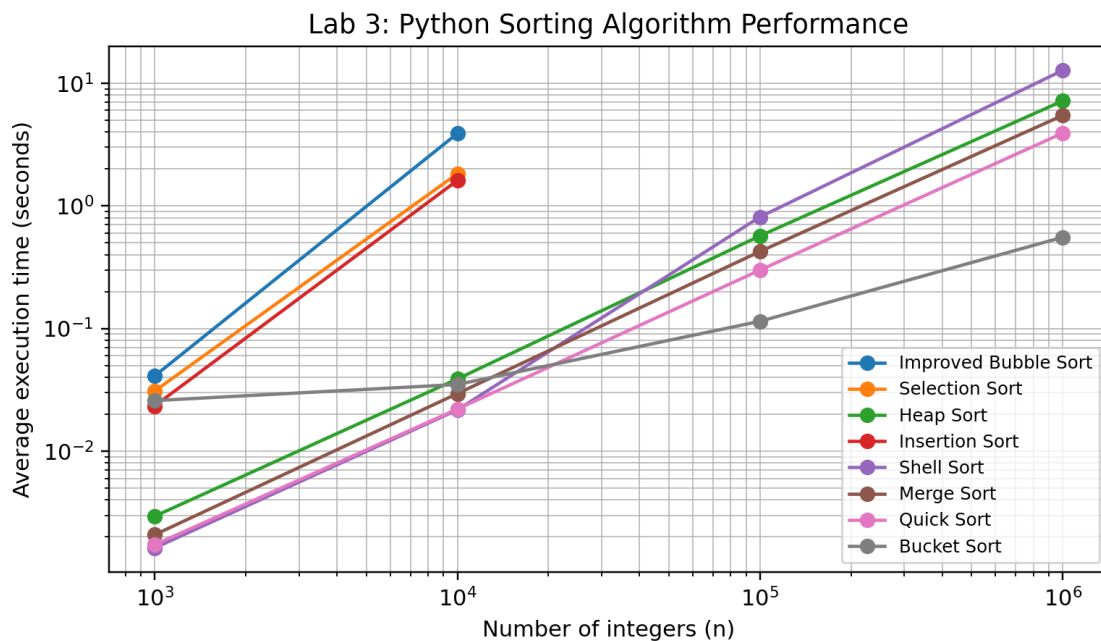


Figure 3. Lab 3 comparison of all required algorithms in Python.

Algorithm at 1,000,000 values	C avg. (s)	Java avg. (s)	Python avg. (s)
Bucket Sort	0.064397	0.102154	0.554806
Quick Sort	0.193603	0.394407	3.920733
Merge Sort	0.284329	0.583542	5.463462
Heap Sort	0.255569	0.459720	7.180757
Shell Sort	0.316123	0.632324	12.731545

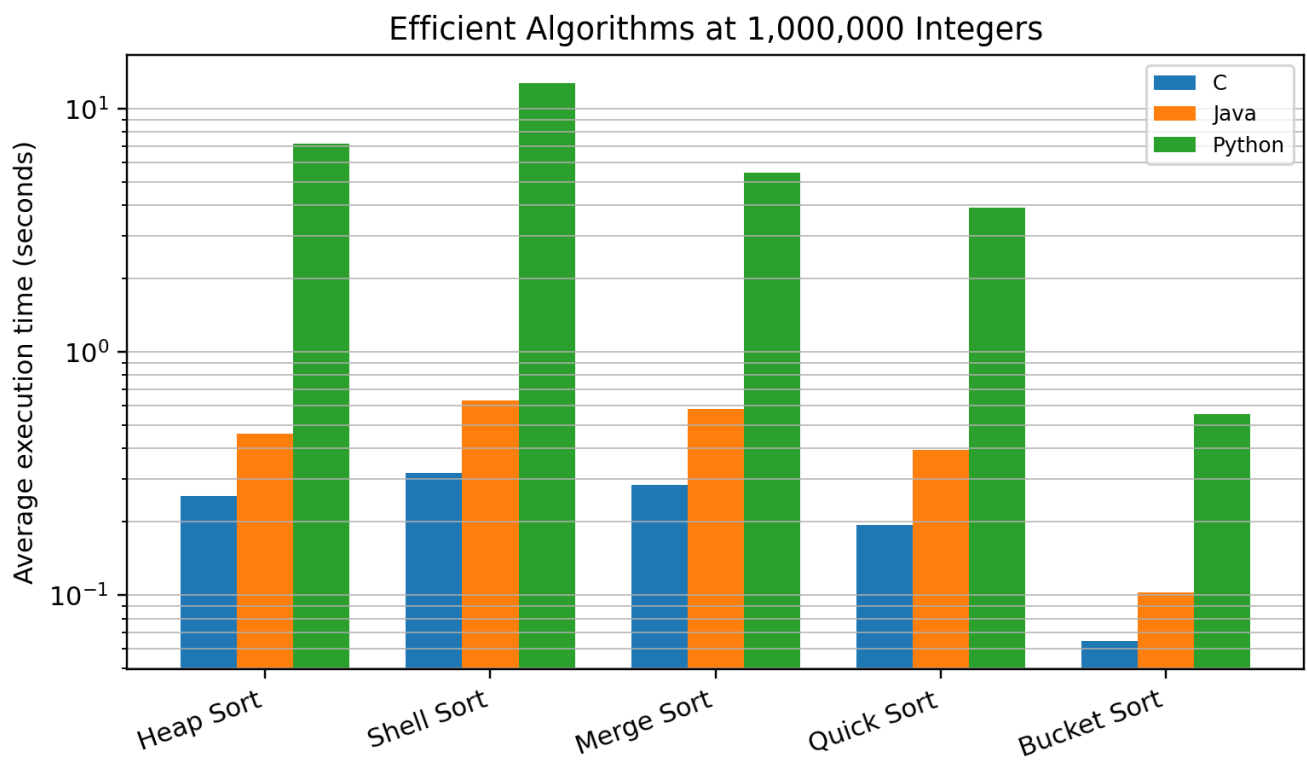


Figure 4. Efficient algorithms at 1,000,000 integers across the three languages.

8. Programming Language Comparison

The same algorithm can perform differently depending on the programming language. C is usually fastest because it compiles directly to machine code and has low runtime overhead. Java can be close to C after the JVM optimizes hot code, but it still has startup and memory-management overhead. Python is much slower for manual loop-based algorithms because each loop operation is interpreted at a higher level.

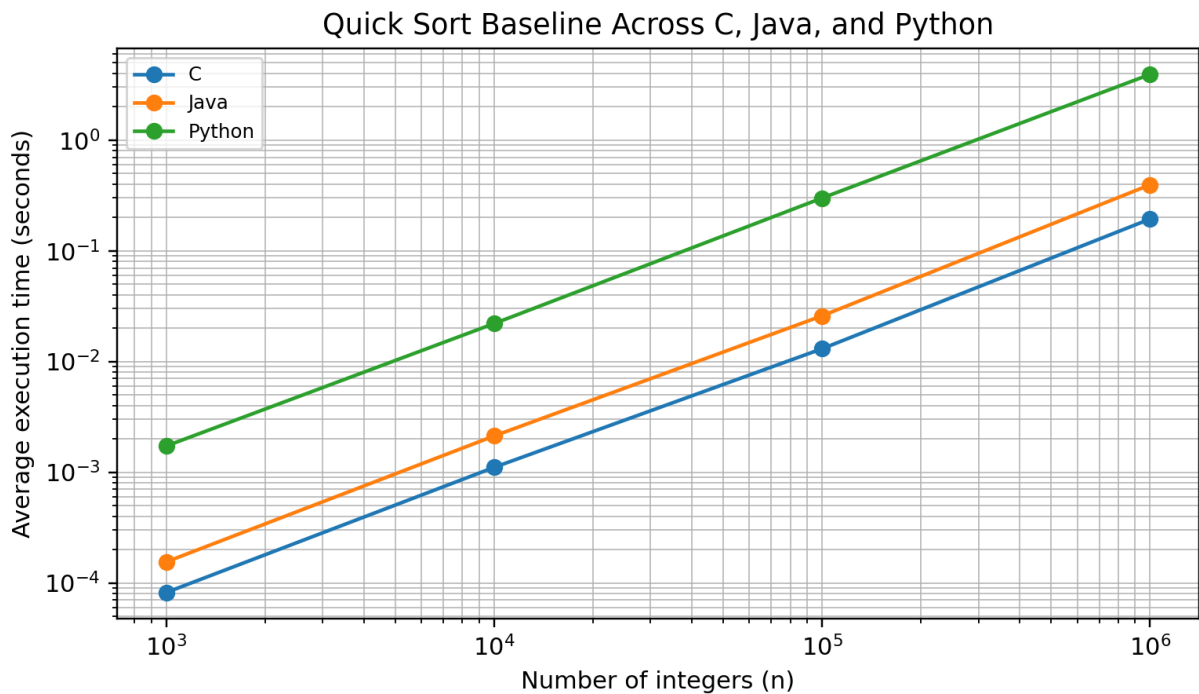


Figure 5. Quick sort baseline across C, Java, and Python.

This language difference is most visible in algorithms that perform many inner-loop comparisons, such as bubble sort, selection sort, and insertion sort. For example, Python may be acceptable for 1,000 or 10,000 values, but it becomes unsuitable for $O(n^2)$ sorting at 100,000 or 1,000,000 values. This is why algorithm choice is often more important than language choice, but language choice still affects the final execution time.

9. Timeout and Practicality Log

Some algorithm/data-size combinations are not practical to run. For random data, $O(n^2)$ algorithms grow too quickly. For example, increasing from 10,000 to 100,000 values increases n by 10, but $O(n^2)$ work increases by about 100. Increasing to 1,000,000 values increases n by 100 compared with 10,000, so $O(n^2)$ work increases by about 10,000.

Algorithm	Data size	Status
Bubble Sort	100,000	Logged for C, Java, and Python
Bubble Sort	1,000,000	Logged for C, Java, and Python
Improved Bubble Sort	100,000	Logged for C, Java, and Python
Improved Bubble Sort	1,000,000	Logged for C, Java, and Python
Insertion Sort	100,000	Logged for C, Java, and Python
Insertion Sort	1,000,000	Logged for C, Java, and Python
Selection Sort	100,000	Logged for C, Java, and Python
Selection Sort	1,000,000	Logged for C, Java, and Python

This log satisfies the requirement to record cases where an algorithm would take too long or become impractical. Instead of wasting more than 10 minutes per case, the program marks these results and continues with the remaining algorithms.

10. Verification of Correctness

The output of every completed run is checked using a sorted-order validation. The validation confirms that each element is less than or equal to the next element. The programs also save before-and-after samples so that the change in the data can be inspected manually.

Verification step	Purpose
Sorted-order check	Every completed run checks $a[i] \leq a[i+1]$ for all positions.
Same input per algorithm	Each algorithm receives a copy of the same random data for the same data size.
Repeated run consistency	Five repeated timings are recorded, then average and standard deviation are calculated.
Before/after record	CSV files contain the first 20 values before and after sorting for completed cases.
Abnormal result prevention	Incorrectly sorted output is marked as failed instead of accepted.

11. Discussion and Analysis

The main finding is that algorithmic complexity dominates performance as the dataset becomes larger. Bubble sort, selection sort, and insertion sort can be easy to understand, but their $O(n^2)$ behavior makes them unsuitable for large random datasets. They are mainly useful for teaching, very small arrays, or special cases such as nearly sorted input.

Heap sort, merge sort, and quick sort remain practical because their average or guaranteed time is $O(n \log n)$. Intuitively, $n \log n$ means the algorithm does not compare every item with every other item. Instead, it repeatedly reduces the problem size, which keeps the number of operations manageable even at 1,000,000 values.

Bucket sort is the fastest in the reference results for very large data because the value range is fixed from 0 to 1,000,000. It does not compare elements one by one like comparison-based sorting. Instead, it counts how many times each value appears, then reconstructs the sorted list. The tradeoff is extra memory proportional to the value range.

Among the general-purpose comparison sorts, quick sort is the best baseline in this experiment. Merge sort is stable and predictable but requires $O(n)$ extra memory. Heap sort has $O(1)$ extra space and a strong worst-case bound, but it is often slower than quick sort in practice because heap operations jump around the array more often.

12. AI Collaboration Process and Improvements Made

AI assistance was used to organize the experiment structure, generate initial implementations of the sorting algorithms, prepare the benchmark framework, and draft this documentation. The generated code was reviewed against the lab requirements to ensure that it includes the required algorithms, data sizes, repeated runs, timeout logging, before/after samples, and charts.

Important improvements were made to reduce common errors. For example, the benchmark copies the original random data before each run so that one algorithm does not receive an already sorted array. The programs also verify sorted output after every run, and the report flags impractical $O(n^2)$ cases instead of presenting unrealistic timing data. These checks help avoid abnormal results such as quick sort appearing slower than bubble sort because of a coding or measurement mistake.

13. Conclusion

The experiment confirms that simple $O(n^2)$ algorithms become impractical when the number of random integers becomes large. Improved bubble sort is better than ordinary bubble sort when the data is already nearly sorted, but it still has $O(n^2)$ average behavior on random data. Selection sort also remains slow because it must repeatedly scan for the next minimum value.

For large datasets, quick sort, merge sort, heap sort, shell sort, and bucket sort are much more practical. Quick sort is the strongest general baseline in this experiment, while bucket sort is fastest when the integer range is limited. Overall, the best algorithm depends on the data characteristics: use quick sort for general speed, merge sort for stable sorting, heap sort when extra memory should be low, and bucket sort when the value range is known and limited.

Appendix A. Files Included in the Submission Package

File	Description
python/sorting_benchmark.py	Python implementation and benchmark.
c/sorting_benchmark.c	C implementation and benchmark.
java/SortingBenchmark.java	Java implementation and benchmark.
results/combined_reference_results.csv	Combined reference/sample timing results used for the report charts.
results/combined_before_after_samples.csv	Before/after sample records.
charts/*.png	Comparison charts included in this report.
run_all_windows.bat	Windows batch file to compile and run all three language versions.